

Hibernate

Q. What is hibernate?

Hibernate is ORM tool of Java which is used for work with database like as JDBC.

Hibernate is an open-source framework that makes it easier to store information in a relational database. It converts Java objects to corresponding database tables easily with ORM. It makes automatic database operations by CRUD (Create, Read, Update, Delete) operations. Thus no need to write SQL queries as Hibernate provides a seamless mechanism and better interacting features.

Q. What is ORM?

ORM stands for Object Relationship Mapping

ORM is design pattern where java class considers as table in database class object consider as row in database table and object field consider as column in database.

ORM, which stands for *Object-Relational Mapping* acts as a translator that converts the Java object into the relational database. It makes it easier to store and retrieve information on Java objects, by using the ORM technique which reduces a lot of manual work.

Q. Why use Hibernate if we have JDBC already?

1. hibernate is database independent framework

because Hibernate use the HQL as query language

HQL stands hibernate query language and it is object oriented version of SQL

HQL Query is generate SQL syntax internally as per the database requirement just we required to configure dialect as per the database.

2. Hibernate create database table automatically programmer not need to create database manually by using create command in database or by programmatically

4. Hibernate provide the cache level support which help us to improve the performance of application.

5. Hibernate provide easy support to joins and association

Etc.

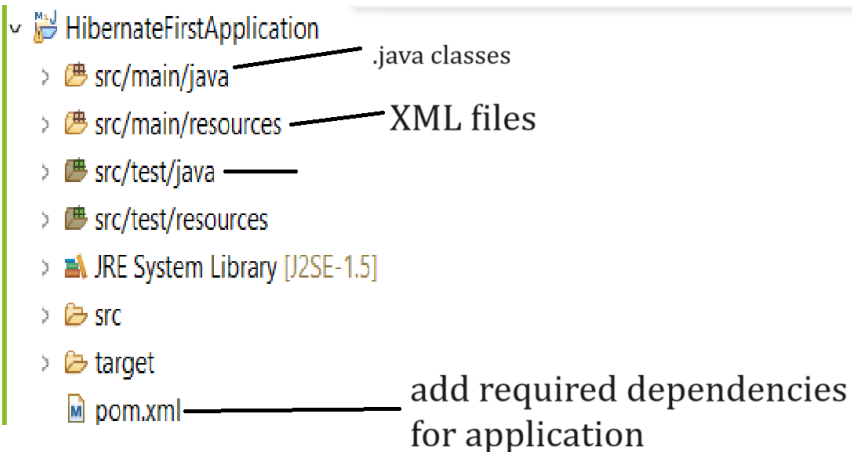
Advantages of using Hibernate?

1. Hibernate prevents data corruption as it ensures that the changes to the database are completely applied.
2. Hibernate provides its query language (HQL) which is similar to SQL.
3. HQL speeds up the performance and generates automatic database tables based on the Java class structure.

How to create Application by using hibernate

If we want to create application using hibernate we have some important steps.

1. Create Maven project using eclipse



Once we create maven project we required to add the following dependencies

a. hibernate core maven dependencies

```
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>5.6.15.Final</version>
</dependency>
```

b. mysql connector maven dependencies

```
<dependencies>
  <!-- https://mvnrepository.com/artifact/org.hibernate/hibernate-core -->
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>5.6.15.Final</version>
</dependency>
  <!-- https://mvnrepository.com/artifact/mysql/mysql-connector-java -->
```

```
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>8.0.33</version>
</dependency>
</dependencies>
```

2. Create POJO class or Entity class

Example: we want to create class name as Employee with field id, name and salary and store its record in database table.

```
package org.techhub;
public class Employee {
  private int id;
  private String name;
  public int getId() {
    return id;
  }
  public void setId(int id) {
    this.id = id;
  }
  public String getName() {
    return name;
  }
  public void setName(String name) {
    this.name = name;
  }
  public int getSal() {
    return sal;
  }
  public void setSal(int sal) {
    this.sal = sal;
  }
  private int sal;
}
```

3. Write hibernate mapping xml file for provide mapping between Java class and Database table

Note: if you want to map your java class with database table we have two ways

- i) Using XML configuration
- ii) Using Annotations.

Now we want to configure hibernate application by using XML here

Steps to map java class with database using XML configuration

a) Create mapping XML file under src/main/resources folder with extension .hbm.xml

Note: class name and file name should be same with .hbm.xml extension

b) Write doctype for mapping file

Q. What is doctype?

Doctype stands document type basically XML is custom tag development language means user can design own tag using XML so if we think about hibernate we have some inbuilt tag which help us to map our java class with database table but these tag design by hibernate development team and we want reuse that tag so we want to use doctype for mapping tags

Q. How to write doctype for hibernate mapping file?

```
<!DOCTYPE hibernate-mapping PUBLIC
"-//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
```

Once we write the doctype you write tags for provide mapping between java class and database table.

Syntax:

```
<hibernate-mapping>
  <class name="classname" table="tablename">
    <id name="propertyname" column="primary key col name"/>
    <property name="propertyname" column="columnname"/>
  </class>
</hibernate-mapping>
```

<hibernate-mapping>: this tag help us to provide mapping between java class and database table.

<class name="classname" table="tablename">: this tag help us map particular class with particular database table

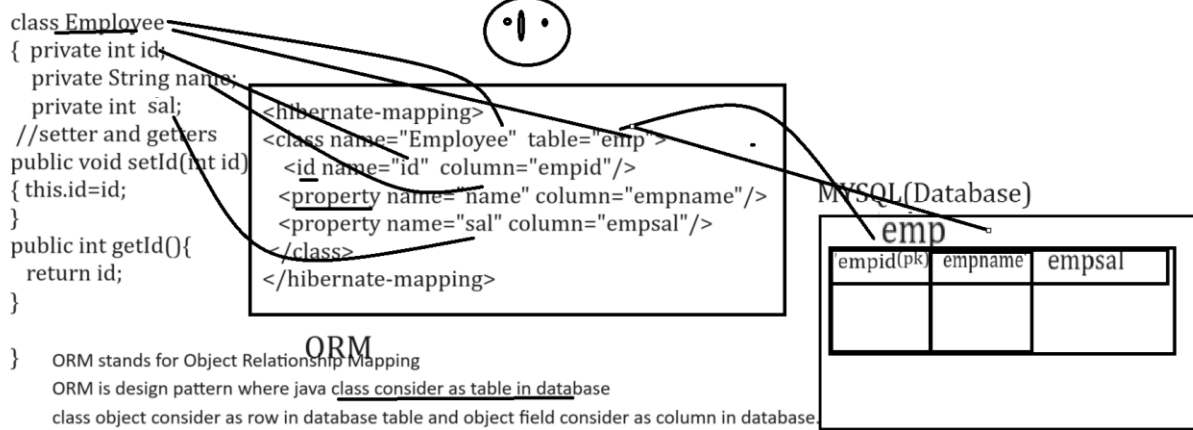
Note: if we not use table attribute in class tag then hibernate by default create table with classname

<id name="propertyname" column="primary key col name"/>:

here id tag indicate primary key column and name attribute indicate field/variable in class which we want to mark as primary key column in database table

<property name="propertyname" column="columnname"/>:

here property tag indicate the non primary key column
name attribute indicate variable which we want to use as column in table and column attribute indicate column name in database table.



employee.hbm.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
    <class name="org.techhub.Employee" table="emp">
        <id name="id" column="empid"/>
        <property name="name" column="empname"/>
        <property name="sal" column="empsal"/>
    </class>
</hibernate-mapping>
```

4. Write configuration file for provide database configuration

If we want to establish connection between java application with database using hibernate we required configuration file provide database configuration as well as hibernate configuration

Note: when we write the configuration file then its extension should be `.cfg.xml`

Example: hibernate.cfg.xml

Steps to write the configuration file

- a. right click on src/main/resources folder and create hibernate.cfg.xml file
- b. Write doctype for hibernate configuration file.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>

        <!-- Related to the connection START -->
<property name="connection.driver_class">com.mysql.cj.jdbc.Driver</property>
        <property
name="connection.url">jdbc:mysql://localhost:3306/hibmayjunejuly</property>
        <property name="connection.user">root</property>
        <property name="connection.password">root</property>
        <!-- Related to the connection END -->
        <!-- Related to hibernate properties START -->
        <property name="show_sql">true</property>
        <property name="dialect">org.hibernate.dialect.MySQL8Dialect</property>
        <property name="hbm2ddl.auto">update</property>
        <!-- Related to hibernate properties END -->
        <!-- Related to mapping START -->
        <mapping resource="employee.hbm.xml" />
    </session-factory>
</hibernate-configuration>
<property name="show_sql">true</property>: this property help us to display the SQL Query
on program console or output screen generate by hibernate container internally
```

<property name="dialect">org.hibernate.dialect.MySQL8Dialect</property>: here dialect are the some inbuilt classes present in hibernate which contain logic for generate SQL syntax as per the database use in backend means now we are using MySQL8Dialect means here our hibernate container generate the SQL syntax as per the mysql8 version of database.

<property name="hbm2ddl.auto">update</property>: this property help us to create new table if not exist in database or update new table if exist in database or drop table from database

here hbm stands for hibernate 2 ddl stands data definition language means if use the update in this property meaning is if your table not present in database then create new table and if present then update it.

<mapping resource="employee.hbm.xml" />: here we have to specify the mapping file name where we map our java class with database table.

5. Write DAO class for work with Database

DAO class means where we write DB logics

Steps to work with DAO class

1. Add org.hibernate.* and org.hibernate.cfg.* package

In this package contain all classes which required to create hibernate and call configuration as well as it help us to create session object and transaction object also.

```
import org.hibernate.*;
```

```
import org.hibernate.cfg.*;
```

2. Create Configuration class and call configuration file

If we want to configuration file we have Configure class and this class provide configure () method to us which help us to call configuration file.

```
package org.techhub;
import org.hibernate.*;
import org.hibernate.cfg.*;
public class EmployeeDAOAPP {
    public static void main(String[] args) {
        Configuration cfg=new Configuration();
        cfg.configure("hibernate.cfg.xml");
    }
}
```

3. Create Reference of SessionFactory

SessionFactory is hibernate container so we can say hibernate container is inbuilt API which is used for manage the overall activity of hibernate like as creating session between java application and database application as well as manage the second level cache etc.

If we want to create reference of SessionFactory we have buildSessionFactory() method of Configuration class and this method return reference of SessionFactory

Syntax: SessionFactory ref=config.buildSessionFactory();

```
package org.techhub;
import org.hibernate.*;
import org.hibernate.cfg.*;
public class EmployeeDAOAPP {
    public static void main(String[] args) {
        Configuration cfg=new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf=config.buildSessionFactory();
    }
}
```

4. Create Reference of Session

If we think about session then in the case of hibernate session indicate communication between java application and database application or it is connection between java application and database application.

So if we want to create session we have Session interface and for creating reference of Session we have openSession() method of SessionFactory and this method return reference of Session interface.

Syntax:

Session ref=factoryref.openSession();

Sample code

```
package org.techhub;
import org.hibernate.*;
import org.hibernate.cfg.*;
public class EmployeeDAOAPP {
    public static void main(String[] args) {
        Configuration cfg=new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf=config.buildSessionFactory();
        Session session=sf.openSession();
    }
}
```

5. Create reference of Transaction

Q. What is Transaction?

Transaction means task perform between particular session called as Transaction like as if we insert record in database called as transaction , if we delete record from database table called as transaction etc.

Means in the case of hibernate we must be commit transaction if we want to reflect your operation in database table.

So if you want to create reference of Transaction we have to method name as beginTransaction() method of Session interface

Syntax: Transaction t=sessionref.beginTransaction();

Sample code

```
package org.techhub;
import org.hibernate.*;
import org.hibernate.cfg.*;
public class EmployeeDAOAPP {
    public static void main(String[] args) {
        Configuration cfg=new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf=cfg.buildSessionFactory();
        Session session=sf.openSession();
        Transaction t=session.beginTransaction();

    }
}
```

6. Create object of entity class

```
package org.techhub;
import org.hibernate.*;
import org.hibernate.cfg.*;
public class EmployeeDAOAPP {
    public static void main(String[] args) {
        Configuration cfg=new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf=cfg.buildSessionFactory();
        Session session=sf.openSession();
        Transaction t=session.beginTransaction();
        Employee emp = new Employee();
```

```

        emp.setId(1);
        emp.setName("ABC");
        emp.setSal(10000);
    }
}

```

7. Work with database

If we want to work with database session provide some inbuilt method to us

void save(Object): this method is used for store data in database table or store java object as row in database table.

Object load(Entityclassname.class, Object primary key): this method can access data from database table using its primary key and return row data in the object of class just we need to convert Object class reference in to entity class name and get data and if primary key is not available in database then raise ObjectNotFoundException at run time.

Object get(Entityclassname.class, Object primarykey): this method can return object data from database table using its key and if key not found return null value.

void saveOrUpdate(Object): this method is used for save the object as row in table if not present and if present update it.

void delete(Object): this method is used for delete the object from table or row from database table.

void update(Object): this method is used for update the object or row in database table.

void cancelQuery()

Cancel the execution of the current query.

void clear()

Completely clear the session.

Connection close()

End the session by releasing the JDBC connection and cleaning up.

Criteria createCriteria(Class persistentClass)

Create a new Criteria instance, for the given entity class, or a superclass of an entity class.

Criteria createCriteria(String entityName)

Create a new Criteria instance, for the given entity name.

Serializable getIdentifier(Object object)

Return the identifier value of the given entity as associated with this session.

Query createFilter(Object collection, String queryString)

Create a new instance of Query for the given collection and filter string.

Query createQuery(String queryString)

Create a new instance of Query for the given HQL query string.

SQLQuery createSQLQuery(String queryString)

Create a new instance of SQLQuery for the given SQL query string.

void delete(Object object)

Remove a persistent instance from the datastore.

void delete(String entityName, Object object)

Remove a persistent instance from the datastore.

Session get(String entityName, Serializable id)

Return the persistent instance of the given named entity with the given identifier, or null if there is no such persistent instance.

SessionFactory getSessionFactory()

Get the session factory which created this session.

void refresh(Object object)

Re-read the state of the given instance from the underlying database.

Transaction getTransaction()

Get the Transaction instance associated with this session.

boolean isConnected()

Check if the session is currently connected.

boolean isDirty()

Does this session contain any changes which must be synchronized with the database?

boolean isOpen()

Check if the session is still open.

Serializable save(Object object)

Persist the given transient instance, first assigning a generated identifier.

void saveOrUpdate(Object object)

Either save(Object) or update(Object) the given instance.

void update(Object object)

Update the persistent instance with the identifier of the given detached instance.

void update(String entityName, Object object)

Update the persistent instance with the identifier of the given detached instance.

8 .close the session**Example for save object as row in database table**

```
package org.techhub;
import org.hibernate.*;
import org.hibernate.cfg.*;
public class EmployeeDAOAPP {
    public static void main(String[] args) {
        Configuration cfg=new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf=cfg.buildSessionFactory();
        Session session=sf.openSession();
        Transaction t=session.beginTransaction();
        Employee emp = new Employee();
        emp.setId(1);
        emp.setName("ABC");
        emp.setSal(10000);
        session.save(emp);
        t.commit();
        session.close();
        System.out.println("Record save in database table");
    }
}
```

Example: we want to delete the employee by using its id.

```

package org.techhub;
import org.hibernate.*;
import org.hibernate.cfg.*;
public class EmployeeDAOAPP {
    public static void main(String[] args) {
        Configuration cfg=new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf=cfg.buildSessionFactory();
        Session session=sf.openSession();
        Transaction t=session.beginTransaction();
        Employee emp = new Employee();
        emp.setId(3);

        session.delete(emp);
        t.commit();
        session.close();
        System.out.println("Record deleted success");
    }
}

```

Example update record of employee

```

package org.techhub;
import org.hibernate.*;
import org.hibernate.cfg.*;
public class EmployeeDAOAPP {
    public static void main(String[] args) {
        Configuration cfg=new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf=cfg.buildSessionFactory();
        Session session=sf.openSession();
        Transaction t=session.beginTransaction();
        Employee emp = new Employee();
        emp.setId(1);
        emp.setName("PQR");
        emp.setSal(5000);
        session.update(emp);
        t.commit();
        session.close();
        System.out.println("Record deleted success");
    }
}

```

```

    }
}

```

Output

```

mysql> select *from emp;
+----+-----+-----+
| empid | empname | empsal |
+----+-----+-----+
| 1 | ABC | 10000 |
| 2 | MNO | 20000 |
+----+-----+-----+
2 rows in set (0.00 sec)

mysql> select *from emp;
+----+-----+-----+
| empid | empname | empsal |
+----+-----+-----+
| 1 | PQR | 5000 |
| 2 | MNO | 20000 |
+----+-----+-----+
2 rows in set (0.00 sec)

```

Before update

After update

Example: Fetch data from table using get() method

Note: if we use get() method and if we not found data return then we get null value

Example:

```
package org.techhub;
```

```
import org.hibernate.*;
```

```
import org.hibernate.cfg.*;
```

```
public class EmployeeDAOAPP {
```

```
    public static void main(String[] args) {
```

```
        Configuration cfg = new Configuration();
```

```
        cfg.configure("hibernate.cfg.xml");
```

```
        SessionFactory sf = cfg.buildSessionFactory();
```

```
        Session session = sf.openSession();
```

```
        Transaction t = session.beginTransaction();
```

```
        Employee emp = new Employee();
```

```
        Object obj = session.get(Employee.class, new Integer(6));
```

```
        Employee e = (Employee) obj;
```

```
        if (e != null) {
```

```
            System.out.println(e);
```

```
        } else {
```

```
            System.out.println("Data not found");
```

```
        }
```

```
        t.commit();
```

```

        session.close();
        System.out.println("Record deleted success");
    }
}

```

Example to fetch data from table using load() method

If we think about load method this method throw ObjectNotFoundException if data not found in table.

```

package org.techhub;

import org.hibernate.*;
import org.hibernate.cfg.*;

public class EmployeeDAOAPP {
    public static void main(String[] args) {
        Configuration cfg = new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf = cfg.buildSessionFactory();
        Session session = sf.openSession();
        Transaction t = session.beginTransaction();
        Employee emp = new Employee();
        try {
            Object obj = session.load(Employee.class, new Integer(1));
            Employee e = (Employee) obj;
            System.out.println(e);
        } catch (ObjectNotFoundException ex) {
            System.out.println("Record not found");
        }
        t.commit();
        session.close();
    }
}

```

HQL

HQL stands for hibernate query language basically it is object oriented version of SQL. Means in HQL we use class name as replacement of table in query and variable of class or field of class replace as column in query and object replace as row. The benefit of HQL is it is database independent query language.

Means syntax of HQL is common on all databases.

What is HQL?

HQL stands for Hibernate Query Language it allows to expression of database queries using entity and property names, rather than relying on traditional SQL that operates on tables and columns. It uses an object-oriented way to operate with Java entity classes and properties.

HQL queries could be dynamic and flexible which supports the aggregation and joins as well such as SUM, and AVG, and uses joins to combine data from the different tables. HQL supports polymorphic queries, meaning you can write a single query that operates on a superclass and retrieves instances of both the superclass and its subclasses.

Example of HQL Query

Syntax: from class name ref; // it is like as select *from tablename

Example: from Employee emp; //select *from employee in SQL
or if we think partial selection or query then your query like as

Syntax: select ref.fieldname, ref.fieldname from Class name ref;

Example: select e.name, e.email, e.contact from employee e;
//It is like as select name, email, contact from employee in SQL.

How to work with HQL

If we want to execute the HQL Query in hibernate we have to use following steps.

1. Add org.hibernate.* and org.hibernate.cfg.* package in application.

```
import org.hibernate.*;  
import org.hibernate.cfg.*;
```

2. Create reference of Query interface

If we want to create reference of Query interface we have createQuery() method of Session interface.

It is an object oriented representation of Hibernate Query. The object of Query can be obtained by calling the createQuery() method Session interface.

The query interface provides many methods.

Syntax: Query ref = sessionref.createQuery(String hqlStatement);

Methods of Query Interface.....

1. **public int executeUpdate()** is used to execute the update or delete query.
2. **public List list()** returns the result of the relation as a list.
3. **public Query setFirstResult(int rowno)** specifies the row number from where record will be retrieved.
4. **public Query setMaxResult(int rowno)** specifies the no. of records to be retrieved from the relation (table).
5. **public Query setParameter(int position, Object value)** it sets the value to the JDBC style query parameter.
6. **public Query setParameter(String name, Object value)** it sets the value to a named query parameter.

Example with source code

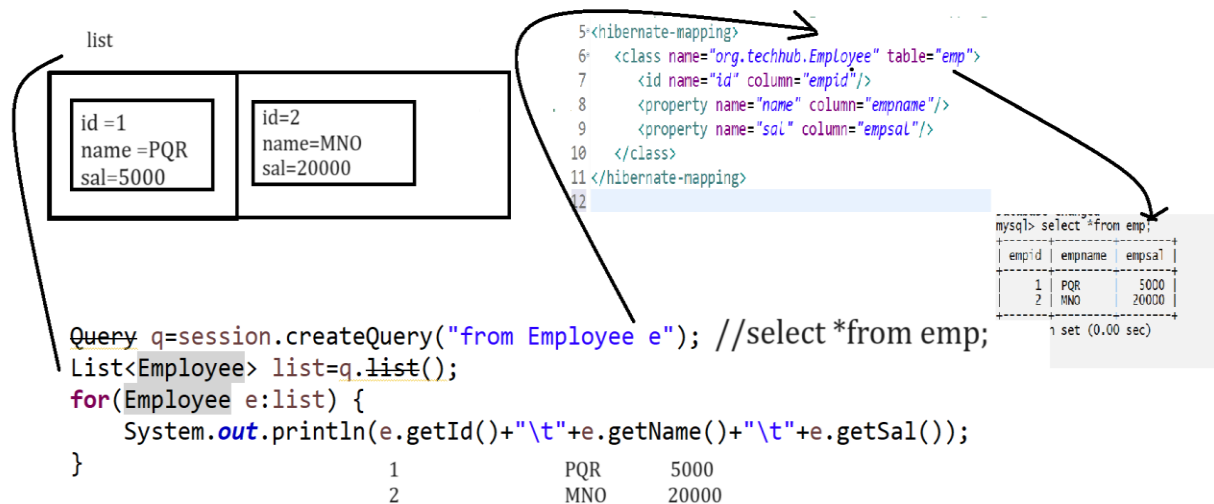
```
package org.techhub;
import org.hibernate.*;
import org.hibernate.cfg.*;
public class HQLQueryApp {
    public static void main(String[] args) {
        Configuration cfg = new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf=cfg.buildSessionFactory();
        Session session=sf.openSession();
        Transaction t=session.beginTransaction();
        //select *from emp; as per our mapping example
        Query q=session.createQuery("from Employee e");
    }
}
```

3. Use its method for execute the HQL Query

If we want to execute the HQL query we have two methods

1. int executeUpdate(): this method help us to execute the all DML statement except select and if statement executed successfully return 1 otherwise return 0.

2. List list(): this method is used for execute the select statement only or select query only so this method help us to fetch row from database and store in collection the form of objects.



Sample code

```

package org.techhub;
import org.hibernate.*;
import org.hibernate.cfg.*;
import java.util.*;
public class HQLQueryApp {
    public static void main(String[] args) {
        Configuration cfg = new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf=cfg.buildSessionFactory();
        Session session=sf.openSession();
        Transaction t=session.beginTransaction();
        //select *from emp; as per our mapping example
        Query q=session.createQuery("from Employee e");
        List<Employee> list=q.list();
        for(Employee e:list) {
            System.out.println(e.getId()+"\t"+e.getName()+"\t"+e.getSal());
        }
    }
}

```

1	PQR	5000
2	MNO	20000

How to fetch partial data from table using HQL

if we want to fetch specified column data from table using HQL we have to use following syntax

Syntax: select ref.variablename, ref.variablename, ref.variablename from class name ref;

Example: suppose consider we have class name as Employee with field id, name and salary and we map this class with emp table in database and we want to fetch only two columns from database table so your HQL query look like as

Select e.name, e.salary from Employee e;

Note: here e work as reference in Employee table so we can fetch data of Employee table by using its reference.

when we fetch partial data from HQL query then we required to List collection with Object class array by using generics here we not fetch complete object of particular class just we can fetch specified columns and column data type may be vary so if we want to store different type of data then we can use object class array

Example with source code

```
package org.techhub;
import org.hibernate.*;
import org.hibernate.cfg.*;
import java.util.*;
public class HQLQueryApp {
    public static void main(String[] args) {
        Configuration cfg = new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf = cfg.buildSessionFactory();
        Session session = sf.openSession();
        Transaction t = session.beginTransaction();
        // select *from emp; as per our mapping example
        Query q = session.createQuery("select e.name,e.sal from Employee e");
        List<Object[]> list = q.list();
        for (Object obj[] : list) {
            System.out.println(obj[0] + "\t" + obj[1]);
        }
    }
}
```

```

    }
}

```

Output

```

PQR      5000
MNO      20000

```

How to pass parameter to HQL query like as PreparedStatement

If we want to pass parameter to HQL Query we have `setParameter ()` method of `Query` interface and we have label concept in HQL for pass parameter here label means we give specified name to HQL parameter

And we pass this name as key in `setParameter ()` function show in following example

Example: Suppose consider we have fetch employee whose salary is 5000.

```

package org.techhub;
import org.hibernate.*;
import org.hibernate.cfg.*;
import java.util.*;
public class HQLQueryApp {
    public static void main(String[] args) {
        Configuration cfg = new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf = cfg.buildSessionFactory();
        Session session = sf.openSession();
        Transaction t = session.beginTransaction();
        // select *from emp; as per our mapping example
        Query q = session.createQuery("select e.name, e.sal from Employee e where
e.sal=:esal");
        q.setParameter("esal", 5000);
        List<Object[]> list = q.list();
        for (Object obj[] : list) {
            System.out.println(obj[0] + "\t" + obj[1]);
        }
    }
}

```

Output

PQR 5000

How to delete record from table using HQL

```
package org.techhub;
import org.hibernate.*;
import org.hibernate.cfg.*;
import java.util.*;
public class HQLQueryApp {
    public static void main(String[] args) {
        Configuration cfg = new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf = cfg.buildSessionFactory();
        Session session = sf.openSession();
        Transaction t = session.beginTransaction();
        Query q=session.createQuery("delete from Employee e where e.id=:eid");
        q.setParameter("eid", 1);
        int value=q.executeUpdate();
        if(value>0) {
            System.out.println("Record Deleted Success....");
        }
        else {
            System.out.println("Record not deleted");
        }
        t.commit();
        session.close();
    }
}
```

How to update record using HQL

We want to update name of employee whose id is 2

```
package org.techhub;
import org.hibernate.*;
import org.hibernate.cfg.*;
import java.util.*;
public class HQLQueryApp {
    public static void main(String[] args) {
```

```

        Configuration cfg = new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf = cfg.buildSessionFactory();
        Session session = sf.openSession();
        Transaction t = session.beginTransaction();
        Query q=session.createQuery("update Employee e set e.name=:ename where
e.id=:eid");
        q.setParameter("eid", 2);
        q.setParameter("ename","PQR");
        int value=q.executeUpdate();
        if(value>0) {
            System.out.println("Record Updated Success....");
        }
        else {
            System.out.println("Record not deleted");
        }
        t.commit();
        session.close();
    }
}

```

How to configure hibernate application by using annotation

If we want to map your java class with database table we have to use following basic annotations.

@Entity: @Entity annotation helps us to represent class

@Table: @Table annotation helps us to map your java class with database table.

@Id: we can use this annotation with field which we want to mark as primary key

@Column: we can use this annotation with field for map with column name.

Example: we want to create player class map with database table.

```

@Entity
@Table(name="playertab")
class Player
{
    @Id
    @Column(name="playerid")
    private int id;
    @Column(name="playername")
    private String name;

    @Column(name="playersal")
    private int sal;

    //setter and getters
}

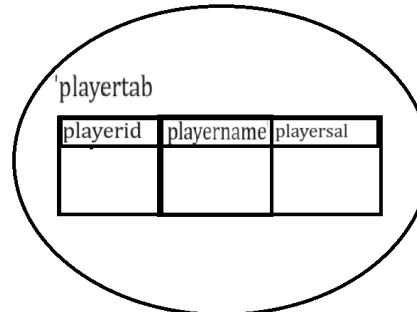
```

```

<class name="Player" table="playertab">
  <id name="id" column="playerid"/>
  <property name="name" column="playername"/>
  <property name="sal" column="playersal"/>
</class>

```

Database table



Steps to Create Hibernate application by using annotation

1. Create Maven project

2. Add following dependencies

<dependencies>

```

<!-- https://mvnrepository.com/artifact/org.hibernate/hibernate-core -->

```

```

<dependency>

```

```

    <groupId>org.hibernate</groupId>

```

```

    <artifactId>hibernate-core</artifactId>

```

```

    <version>5.6.15.Final</version>

```

```

</dependency>

```

```

<!-- https://mvnrepository.com/artifact/mysql/mysql-connector-java -->

```

```

<dependency>

```

```

    <groupId>mysql</groupId>

```

```

    <artifactId>mysql-connector-java</artifactId>

```

```

    <version>8.0.33</version>

```

```

</dependency>

```

```

</dependencies>

```

3. Create Entity class and Map with database table.

```

package org.techhub;

```

```

import javax.persistence.Column;

```

```

import javax.persistence.Entity;

```

```

import javax.persistence.Id;

```

```

import javax.persistence.Table;

```

```

@Entity

```

```

@Table(name="playertab")

```

```

public class Player {

```

```

@Id
@Column(name="playerid")
private int id;

public int getId() {
    return id;
}

public void setId(int id) {
    this.id = id;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public int getSal() {
    return sal;
}

public void setSal(int sal) {
    this.sal = sal;
}

@Column(name="plyaername")
private String name;
@Column(name="playersal")
private int sal;
}

```

4. Create Hibernate configuration file and configure entity class

When we use annotation then we required configure your entity class as mapping class in configuration file.

Syntax: <mapping class="packagename.classname" />

Example with source code


```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <!-- Related to the connection START -->
        <property name="connection.driver_class">com.mysql.cj.jdbc.Driver</property>
        <property
name="connection.url">jdbc:mysql://localhost:3306/hibmayjunejuly</property>
        <property name="connection.user">root</property>
        <property name="connection.password">root</property>
        <!-- Related to the connection END -->
        <!-- Related to hibernate properties START -->
        <property name="show_sql">true</property>
        <property name="dialect">org.hibernate.dialect.MySQL8Dialect</property>
        <property name="hbm2ddl.auto">update</property>
        <!-- Related to hibernate properties END -->
        <!-- Related to mapping START -->
        <mapping class="org.techhub.Player"/>
    </session-factory>
</hibernate-configuration>

```

4. Write DAO class and Write database logics

```

package org.techhub;
import org.hibernate.*;
import org.hibernate.cfg.*;
public class DAOAPP {

    public static void main(String[] args) {
        Configuration cfg=new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf=cfg.buildSessionFactory();
        Session session=sf.openSession();
        Transaction t=session.beginTransaction();
        Player p1 = new Player();
        p1.setId(1);
        p1.setName("ABCD");
    }
}

```

```

        p1.setSal(10000);
        session.save(p1);
        t.commit();
        session.close();
        System.out.println("Record Save Success.....");
    }
}

```

Generator in hibernate

Generators are the algorithm in hibernate or generator are the some inbuilt classes in hibernate which contain algorithm which help us to generate the primary key column automatically in database table using hibernate

A generator class is used to generate an ID for the for an object, which is going to be the primary key of the database table. All generator classes implement **org.hibernate.id.IdentifierGenerator** interface. One can create their own generator class by implementing the above-mentioned interface and overriding the **generator(SharedSessionContractImplementor sess, Object obj)** method.

How to use generator using XML

If we want to use the generator classes using hibernate we have

Following syntax

```

<class name="classname" table="tablename">
  <id name="fieldname" column="columnname">
    <generator class="classname"/>
  </id>
</class>

```

Types of generator in hibernate

- **assigned** – This generator indicates that the application will assign the primary key value. Hibernate doesn't generate any value in this case.
- **identity** – This generator uses the database's auto-increment feature to generate primary key values. It works with most databases and is suitable for simple use cases. Oracle does not support identity generator. MySQL, MS SQL Server, DB2, etc. supports this.

- **sequence** – This generator uses a database sequence to generate primary key values. It provides better performance and control compared to identity in some cases.

Command to create a sequence

```
create sequence <sequence_name> start with <number> increment by <number>
```

Note – MySQL does not support sequences. Oracle does support sequences.

Entry in employee.hbm.xml

```
<hibernate-mapping>
  <class name="com.mypackage.Employee" table="emp">
    <id name="id">
      <generator class="sequence">
        <param name="sequence">datasource_name</param>
      </generator>
    </id>
  </hibernate-mapping>
```

- **increment** – This generator generates primary key values by incrementing a value stored in memory.
- **hilo** – This generator uses a high-low algorithm to generate primary key values. It combines the advantages of sequence and increment.
- **uuid** – This generator generates universally unique identifiers (UUIDs) as primary key values. It is useful for distributed systems where unique IDs are required.
- **native** – This generator delegates the primary key generation strategy to the underlying database. If identity is supported by the underlying database, it chooses identity. Otherwise, it chooses sequence or hilo. It chooses the best strategy based on the database dialect.
- **foreign** – This generator uses the primary key value from another associated entity as the primary key value for the current entity.

1. Assigned: assigned generator is used for assign primary key value manually by programmer and it is default generator class of hibernate

Example: player.hbm.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
  "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
  "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
  <class name="org.techhub.Player" table="playertab">
```

```
<id name="id" column="playerid">
  <generator class="assigned"></generator>
</id>
<property name="name" column="playername"/>
<property name="sal" column="playersal"/>
</class>
</hibernate-mapping>
```

Player.java

```
package org.techhub;
```

```
import javax.persistence.Column;
```

```
import javax.persistence.Entity;
```

```
import javax.persistence.Id;
```

```
import javax.persistence.Table;
```

```
public class Player {
```

```
    private int id;
```

```
    public int getId() {
        return id;
    }
```

```
    public void setId(int id) {
        this.id = id;
    }
```

```
    public String getName() {
        return name;
    }
```

```
    public void setName(String name) {
        this.name = name;
    }
```

```
    public int getSal() {
        return sal;
    }
```

```

    public void setSal(int sal) {
        this.sal = sal;
    }

    private String name;

    private int sal;

}

DAOAPP
package org.techhub;
import org.hibernate.*;
import org.hibernate.cfg.*;
public class DAOAPP {

    public static void main(String[] args) {
        Configuration cfg=new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf=cfg.buildSessionFactory();
        Session session=sf.openSession();
        Transaction t=session.beginTransaction();
        Player p1 = new Player();
        p1.setId(3);
        p1.setName("STV");
        p1.setSal(20000);
        session.save(p1);
        t.commit();
        session.close();
        System.out.println("Record Save Success.....");
    }

}

```

2. Sequence : sequence generator is database dependent generator and when we use sequence generator then hibernate create hibernate_sequence in database and generate primary key value from that sequence.

player.hbm.xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```

<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
    <class name="org.techhub.Player" table="player">
        <id name="id" column="playerid">
            <generator class="sequence"></generator>
        </id>
        <property name="name" column="playername"/>
        <property name="sal" column="playersal"/>
    </class>
</hibernate-mapping>

```

Player.java

```

package org.techhub;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;
public class Player {

    private int id;

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}

```

```

        public int getSal() {
            return sal;
        }

        public void setSal(int sal) {
            this.sal = sal;
        }

        private String name;

        private int sal;
    }
}

DAOAPP.java
package org.techhub;
import org.hibernate.*;
import org.hibernate.cfg.*;
public class DAOAPP {

    public static void main(String[] args) {
        Configuration cfg=new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf=cfg.buildSessionFactory();
        Session session=sf.openSession();
        Transaction t=session.beginTransaction();
        Player p1 = new Player();

        p1.setName("STV");
        p1.setSal(20000);
        session.save(p1);
        t.commit();
        session.close();
        System.out.println("Record Save Success.....");
    }
}

```

3. Increment: it is database independent generator and it is used generate primary key by using logic as

select max(primarykeycol)+1 from tablename ;

Means this generate take the max value of primary key and + 1 it and store in database table and if first time record not found in table then consider first primary key value max as 0 then +1

refer code from previous xml I am just updating player.hbm.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
    <class name="org.techhub.Player" table="player">
        <id name="id" column="playerid">
            <generator class="increment"></generator>
        </id>
        <property name="name" column="playername"/>
        <property name="sal" column="playersal"/>
    </class>
</hibernate-mapping>
```

4. Native: this generator help us to generate the primary key by using default primary key generation technique of database means if we use mysql then the default primary key generation technique is auto_increment and if use the oracle then sequence

player.hbm.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
    <class name="org.techhub.Player" table="player">
        <id name="id" column="playerid">
            <generator class="native"></generator>
        </id>
        <property name="name" column="playername"/>
        <property name="sal" column="playersal"/>
    </class>
```


</hibernate-mapping>

5. UUID: stands for Universally Unique Identifier this generator help us generate primary key value using varchar datatype and generate primary by using ip address of computer, Start-up time of JVM, System time, and Counter value in JVM

Example with source code

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
    <class name="org.techhub.Player" table="player">
        <id name="id" column="playerid">
            <generator class="uuid"></generator>
        </id>
        <property name="name" column="playername"/>
        <property name="sal" column="playersal"/>
    </class>
</hibernate-mapping>
```

6. hilo: hilo generator can generate the primary key with difference of 32767

Means first primary key value is 1 then second is 32768 and so on

But hibernate remove hilo generator after hibernate 5

Etc.

How to use generators by using annotation

If we want to use generators using hibernate we have @GeneratedValue annotations and this annotation provide different strategies to us to generate the primary key value

Following strategies to generate the primary key

GenerationType.IDENTITY: this strategy help us to generate primary key value by database itself using auto_increment column option and it relies on database native support to generate unique keys.

Example with source code

Player.java

```
package org.techhub;
```

```
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;
@Entity
@Table(name="playergen")
public class Player {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name="playerid")
    private int id;
    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getSal() {
        return sal;
    }

    public void setSal(int sal) {
        this.sal = sal;
    }

    private String name;
```

```

        private int sal;

    }

```

hibernate.cfg.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <!-- Related to the connection START -->
        <property name="connection.driver_class">com.mysql.cj.jdbc.Driver</property>
        <property
name="connection.url">jdbc:mysql://localhost:3306/hibmayjunejuly</property>
        <property name="connection.user">root</property>
        <property name="connection.password">root</property>
        <!-- Related to the connection END -->
        <!-- Related to hibernate properties START -->
        <property name="show_sql">true</property>
        <property name="dialect">org.hibernate.dialect.MySQL8Dialect</property>
        <property name="hbm2ddl.auto">update</property>
        <!-- Related to hibernate properties END -->
        <!-- Related to mapping START -->
        <mapping class="org.techhub.Player" />
    </session-factory>
</hibernate-configuration>

```

DAOAPP

```

package org.techhub;
import org.hibernate.*;
import org.hibernate.cfg.*;
public class DAOAPP {
    public static void main(String[] args) {
        Configuration cfg=new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf=cfg.buildSessionFactory();
    }
}

```

```

        Session session=sf.openSession();
        Transaction t=session.beginTransaction();
        Player p1 = new Player();
        p1.setName("STV");
        p1.setSal(20000);
        session.save(p1);
        t.commit();
        session.close();
        System.out.println("Record Save Success.....");
    }
}

```

Table create with primary key value like as

create table playergen (playerid integer not null auto_increment, name varchar(255), sal integer not null, primary key (playerid)) engine=InnoDB

GenerationType.AUTO: this is the default strategy and persistence provider which is automatically select appropriate generation strategy as per the database.

Example: generate primary key value using hibernate_sequence technique

Example with source code

Player.java

```

package org.techhub;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;
@Entity
@Table(name="playergen1")
public class Player {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    @Column(name="playerid")
    private int id;
    public int getId() {
        return id;
    }
}

```

```

    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public int getSal() {
        return sal;
    }
    public void setSal(int sal) {
        this.sal = sal;
    }
    private String name;
    private int sal;
}

```

hibernate.cfg.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <!-- Related to the connection START -->
        <property name="connection.driver_class">com.mysql.cj.jdbc.Driver</property>
        <property
name="connection.url">jdbc:mysql://localhost:3306/hibmayiunejuly</property>
        <property name="connection.user">root</property>
        <property name="connection.password">root</property>
        <!-- Related to the connection END -->
        <!-- Related to hibernate properties START -->
        <property name="show_sql">>true</property>
        <property name="dialect">org.hibernate.dialect.MySQL8Dialect</property>
        <property name="hbm2ddl.auto">update</property>
        <!-- Related to hibernate properties END -->
    
```

```

        <!-- Related to mapping START -->
        <mapping class="org.techhub.Player" />
    </session-factory>
</hibernate-configuration>

```

DAOAPP

```

package org.techhub;
import org.hibernate.*;
import org.hibernate.cfg.*;
public class DAOAPP {
    public static void main(String[] args) {
        Configuration cfg=new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf=cfg.buildSessionFactory();
        Session session=sf.openSession();
        Transaction t=session.beginTransaction();
        Player p1 = new Player();
        p1.setName("STV");
        p1.setSal(20000);
        session.save(p1);
        t.commit();
        session.close();
        System.out.println("Record Save Success.....");
    }
}

```

Output:

```

Hibernate: create table playergen1 (playerid integer not null, name varchar(255), sal integer not
null, primary key (playerid)) engine=InnoDB
Hibernate: select next_val as id_val from hibernate_sequence for update
Hibernate: update hibernate_sequence set next_val= ? where next_val=?
Hibernate: insert into playergen1 (name, sal, playerid) values (?, ?, ?)
Record Save Success.....

```

GenerationType.Table: this strategy uses separate database table to generate primary key like as hibernate_sequence table and using that table allocate unique primary key values.

Example with source code

Player.java

```
package org.techhub;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;
@Entity
@Table(name="playergen")
public class Player {
    @Id
    @GeneratedValue(strategy = GenerationType.TABLE)
    @Column(name="playerid")
    private int id;

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getSal() {
        return sal;
    }

    public void setSal(int sal) {
```

```

        this.sal = sal;
    }

    private String name;

    private int sal;
}

```

hibernate.cfg.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <!-- Related to the connection START -->
        <property name="connection.driver_class">com.mysql.cj.jdbc.Driver</property>
        <property
name="connection.url">jdbc:mysql://localhost:3306/hibmayiunejuly</property>
        <property name="connection.user">root</property>
        <property name="connection.password">root</property>
        <!-- Related to the connection END -->
        <!-- Related to hibernate properties START -->
        <property name="show_sql">true</property>
        <property name="dialect">org.hibernate.dialect.MySQL8Dialect</property>
        <property name="hbm2ddl.auto">update</property>
        <!-- Related to hibernate properties END -->
        <!-- Related to mapping START -->
        <mapping class="org.techhub.Player" />
    </session-factory>
</hibernate-configuration>

```

DAOAPP

```

package org.techhub;
import org.hibernate.*;
import org.hibernate.cfg.*;
public class DAOAPP {
    public static void main(String[] args) {

```



```

        Configuration cfg=new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf=cfg.buildSessionFactory();
        Session session=sf.openSession();
        Transaction t=session.beginTransaction();
        Player p1 = new Player();
        p1.setName("STV");
        p1.setSal(20000);
        session.save(p1);
        t.commit();
        session.close();
        System.out.println("Record Save Success.....");
    }
}

```

GenerationType.SEQUENCE: generate primary key by using sequence so user can create own sequence and can generate primary value from that.

HCQL

HCQL stands for hibernate Criteria Query Language it is like as HQL but HCQL specially design for execute the select commands with some specified criteria.

Means using HCQL we can execute query using like operation in SQL, or aggregate function in SQL.

Example: we want to fetch employee whose salary between 1000 and 10000
So this type of query we can execute by using HCQL in hibernate

Example: we want to fetch employee whose name start with a and end with sh

Steps to work with HCQL

1. add org.hibernate.* and org.hibernate.cfg.* and org.hibernate.criterion.* in application

```

import org.hibernate.*;
import org.hibernate.cfg.*;
import org.hibernate.criterion.*;

```

2. Create Reference of Criteria interface

if we want to create reference of Criteria interface we have createCriteria() method of Session

Syntax: Criteria ref=sessionref.createCriteria(classname.class);

Sample code

```
package org.techhub;
import org.hibernate.*;
import org.hibernate.cfg.*;
import org.hibernate.criterion.*;
public class HCQLAPP {
    public static void main(String[] args) {
        Configuration cfg = new Configuration();
        cfg.configure("hiberate.cfg.xml");
        SessionFactory sf=cfg.buildSessionFactory();
        Session session=sf.openSession();
        Transaction t=session.beginTransaction();
        Criteria c=session.createCriteria(Employee.class);
    }
}
```

3. Create Reference of Criterion interface

Criterion is a interface which help us apply restrictions or conditions on Criteria interface and if we want to create reference of Criterion interface we have Restrictions class and it contain some static methods which return reference of Criterion interface.

Restrictions class contain all static methods for apply restrictions on Query like

Criteria ref=Restrictions.gt("fieldname", value): this method help us to return result greater than specified value.

Criteria ref=Restrictions.lt("fieldname", value): this method help us to return result less than with specified value.

Criteria ref=Restrictions.in("fieldname", Object ...): here we can pass n number values which we want to compare

Criteria ref=Restrictions.between("fieldname", int lowvalue, int highvalue): this help us to fetch data between two specified values.

Criteria ref=Restrictions.like("fieldname", String pattern, MatchMode);

Etc.

Parameter details of like function

fieldname: this attribute indicate field name or property name

String pattern: this attribute help us to specify search pattern

MatchMode: this is the class which contain some sub attributes which decide how we can pattern means match pattern with start value or end value or anywhere so it contain some constant values like as

MatchMode.START

MatchMode.END

MatchMode.ANY_WHERE

Example: we want to fetch employee record whose id is greater than 2.

Sample code

```
package org.techhub;
import org.hibernate.*;
import org.hibernate.cfg.*;
import org.hibernate.criterion.*;
public class HCQLAPP {
    public static void main(String[] args) {
        Configuration cfg = new Configuration();
        cfg.configure("hiberate.cfg.xml");
        SessionFactory sf=cfg.buildSessionFactory();
        Session session=sf.openSession();
        Transaction t=session.beginTransaction();
        Criteria c=session.createCriteria(Employee.class);
        Criterion cn=Restrictions.gt("id",2);
    }
}
```

4. Add Criterion interface reference on Criteria

If we want to add Criterion interface reference on Criteria we have add() method of Criteria interface.

Syntax: void add(Criterion): we can apply Criterion reference on Criteria interface using add method shown in following code.

```
package org.techhub;
import org.hibernate.*;
import org.hibernate.cfg.*;
import org.hibernate.criterion.*;
public class HCQLAPP {
    public static void main(String[] args) {
```

```

        Configuration cfg = new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf=cfg.buildSessionFactory();
        Session session=sf.openSession();
        Transaction t=session.beginTransaction();
        Criteria c=session.createCriteria(Employee.class);
        Criterion cn=Restrictions.gt("id",2);
        c.add(cn);
    }
}

```

5. Execute HCQL statement: if we want to execute HCQL statement we have list() method of Query interface

List list(): using this method we can fetch or select data from database table.

Sample code

```

package org.techhub;
import org.hibernate.*;
import org.hibernate.cfg.*;
import org.hibernate.criterion.*;
import java.util.*;
public class HCQLAPP {
    public static void main(String[] args) {
        Configuration cfg = new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf=cfg.buildSessionFactory();
        Session session=sf.openSession();
        Transaction t=session.beginTransaction();
        Criteria c=session.createCriteria(Employee.class);
        Criterion cn=Restrictions.gt("id",2);
        c.add(cn);
        List<Employee> list=c.list();
        for(Employee e:list) {
            System.out.println(e.getId()+"\t"+e.getName()+"\t"+e.getSal());
        }
    }
}

```

Example: we want to fetch employee whose name start with p

```

package org.techhub;

```

```

import org.hibernate.*;
import org.hibernate.cfg.*;
import org.hibernate.criterion.*;
import java.util.*;

public class HCQLAPP {
    public static void main(String[] args) {
        Configuration cfg = new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf=cfg.buildSessionFactory();
        Session session=sf.openSession();
        Transaction t=session.beginTransaction();
        Criteria c=session.createCriteria(Employee.class);
        Criterion cn=Restrictions.like("name","p",MatchMode.START);
        c.add(cn);
        List<Employee> list=c.list();
        for(Employee e:list) {
            System.out.println(e.getId()+"\t"+e.getName()+"\t"+e.getSal());
        }
    }
}

```

Example: we want to fetch employee data whose name ends with Z.

```

package org.techhub;

import org.hibernate.*;
import org.hibernate.cfg.*;
import org.hibernate.criterion.*;
import java.util.*;

public class HCQLAPP {
    public static void main(String[] args) {
        Configuration cfg = new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf=cfg.buildSessionFactory();
        Session session=sf.openSession();
        Transaction t=session.beginTransaction();
        Criteria c=session.createCriteria(Employee.class);
        Criterion cn=Restrictions.like("name","z",MatchMode.END);
        c.add(cn);
        List<Employee> list=c.list();
    }
}

```

```

        for(Employee e:list) {
            System.out.println(e.getId()+"\t"+e.getName()+"\t"+e.getSal());
        }
    }
}

```

Example: we want to fetch data of employee whose salary is 1000 or 20000 or 30000

Example with source code

```

package org.techhub;
import org.hibernate.*;
import org.hibernate.cfg.*;
import org.hibernate.criterion.*;
import java.util.*;
public class HCQLAPP {
    public static void main(String[] args) {
        Configuration cfg = new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf=cfg.buildSessionFactory();
        Session session=sf.openSession();
        Transaction t=session.beginTransaction();
        Criteria c=session.createCriteria(Employee.class);
        Criterion cn=Restrictions.in("sal", 1000,20000,30000);
        c.add(cn);
        List<Employee> list=c.list();
        for(Employee e:list) {
            System.out.println(e.getId()+"\t"+e.getName()+"\t"+e.getSal());
        }
    }
}

```

Example: we want to fetch employee whose salary between 10000 to 30000

```

package org.techhub;
import org.hibernate.*;
import org.hibernate.cfg.*;
import org.hibernate.criterion.*;
import java.util.*;
public class HCQLAPP {
    public static void main(String[] args) {
        Configuration cfg = new Configuration();

```

```

        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf=cfg.buildSessionFactory();
        Session session=sf.openSession();
        Transaction t=session.beginTransaction();
        Criteria c=session.createCriteria(Employee.class);
        Criterion cn=Restrictions.between("sal", 10000,30000);
        c.add(cn);
        List<Employee> list=c.list();
        for(Employee e:list) {
            System.out.println(e.getId()+"\t"+e.getName()+"\t"+e.getSal());
        }
    }
}

```

How to arrange record in ascending order and in descending order

If we want to arrange record in ascending order and in descending order we have Order class and Order class contain two methods

asc() and desc() and this are the static method of Order class which decide how data arrange by HCQL Query.

and if we want to Order on Criteria we have method name as
void addOrder(Order)

Sample code here

```

package org.techhub;
import org.hibernate.*;
import org.hibernate.cfg.*;
import org.hibernate.criterion.*;
import java.util.*;
public class HCQLAPP {
    public static void main(String[] args) {
        Configuration cfg = new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf=cfg.buildSessionFactory();
        Session session=sf.openSession();
        Transaction t=session.beginTransaction();
        Criteria c=session.createCriteria(Employee.class);
        Order o=Order.desc("sal");
        c.addOrder(o);
        List<Employee> list=c.list();
    }
}

```

```

    for(Employee e:list) {
        System.out.println(e.getId()+"\t"+e.getName()+"\t"+e.getSal());
    }
}
}

```

Association & Joins using Hibernate

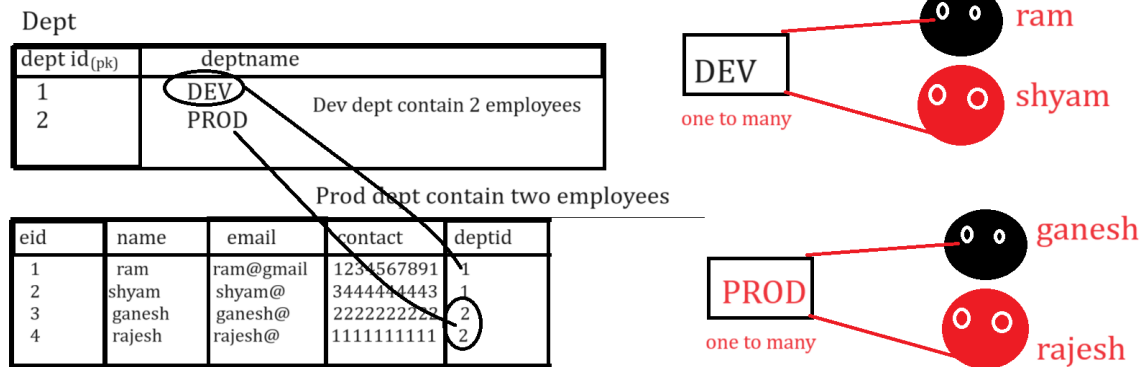
Association means we provide relationship between different tables like as one too many or many to one or many to many or one to one etc.

Types of association

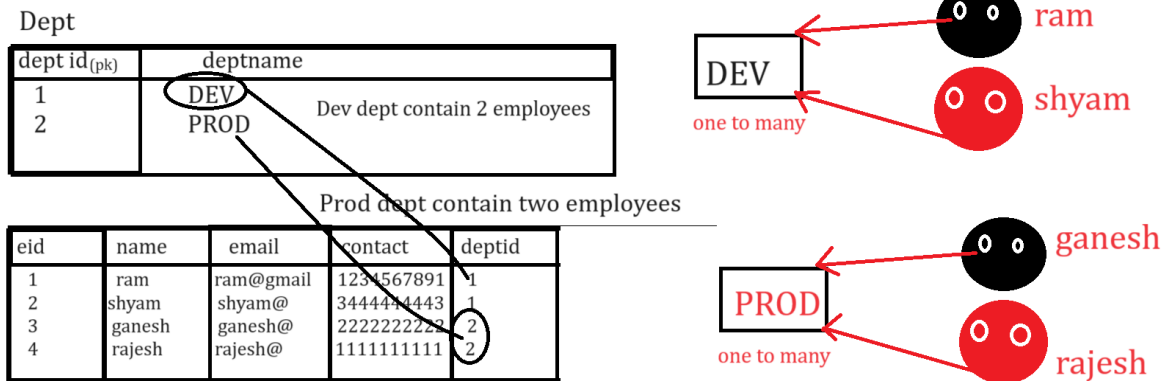
1. One too many: single record can point to more than one record or entity called as one too many associations.

In a one-to-many association, one instance of an entity is associated with multiple instances of another entity. This can be implemented using the `@OneToMany` and `@ManyToOne` annotations.

Example: Single dept can have more than one employee



2. Many to one: many to one association means more than one entity can point to single entity



If we think about above example from employee side then we can say two employees has single dept.

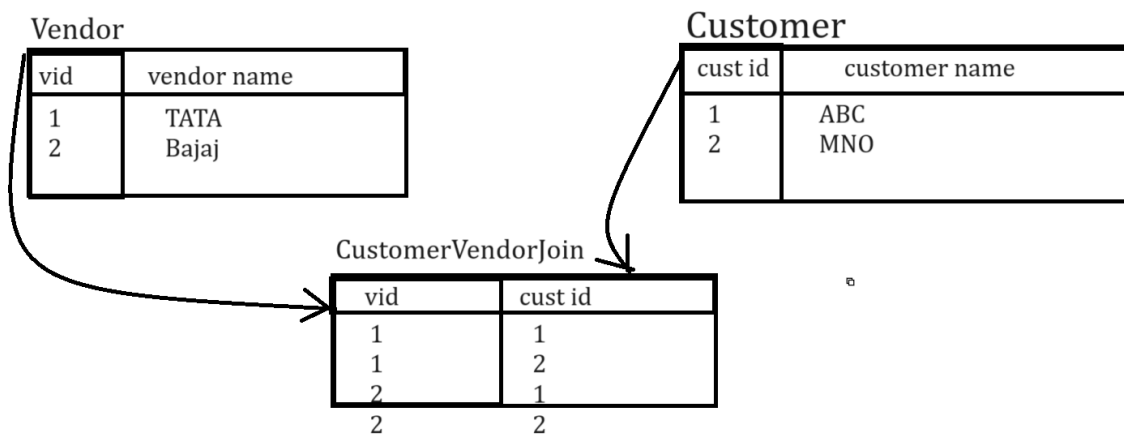
3. Many too many: many too many association is combination of one too many and many to one.

In a many-to-many association, multiple instance of an entity is associated with multiple instances of another entity. This can be implemented using the @ManyToMany annotations.

Example: suppose consider if we think about vendor and customer relation or shop keeper and customer relation.

Single shop keeper can have more than one customer or single customer can have more than one shop keeper.

If we think about many to many relation practically then we required to create one intermediate table which contain only foreign keys only.



4. one to one: one to one relationship means one entity point to single another entity called as one to one relationship

In a one-to-one association, each instance of an entity is associated with one instance of another entity.

Example: suppose consider we have Person and Passport single person can have single passport no

Person

pid	name	email	contact
1	ram	ram@gmail	3344556677
2	shyam	shyam@	2222222222

Passport

ppid	passportnum	pid (unique)
1	Ind123455	1
2	Ind343434	2

How to implement one too many association using hibernate

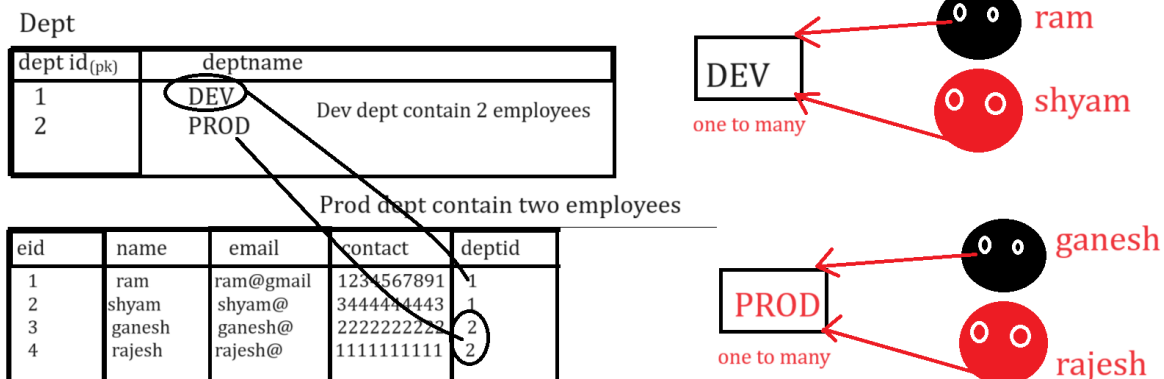
If we want to implement one too many association using hibernate we have two ways

1. Using XML Configuration
2. Using Annotations

How to implement one too many association using XML configuration

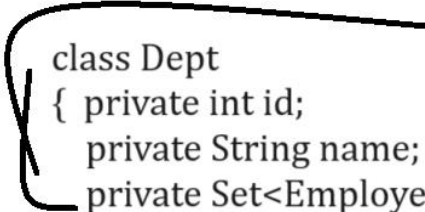
Steps to implement one too many association using XML configuration

We want to implement scenario single dept can have more than one employees



1. Create Parent class with collection property

Suppose consider we think here we have parent class name as Dept because in our diagram we have parent table name as Dept as per db



```
class Dept
{ private int id;
  private String name;
  private Set<Employee> employeeSet;

  //setters and getters

}
```

Dept has set of employees

Example with source code

```
package org.techhub.onetomany;

public class Dept {
    private int id;
    private String name;
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
}
```

2. Create Child class with one extra field: Here we have child class name as Employee because employee table use the foreign reference from Dept table so employee is our child table.

Example with source code

```
package org.techhub.onetomany;

public class Employee {
    private int id;
    private String name;
    public int getId() {
        return id;
    }
}
```

```

    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public int getDeptid() {
        return deptid;
    }
    public void setDeptid(int deptid) {
        this.deptid = deptid;
    }
    private int deptid;
}

```

3. Configure the parent class using xml with database table

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
    <class name="org.techhub.onetomany.Dept" table="dept">
        <id name="id" column="deptid"></id>
        <property name="name" column="deptname"/>
        <set name="employeeSet" cascade="all">
            <key column="deptid"></key>
            <one-to-many class="org.techhub.onetomany.Employee"/>
        </set>
    </class>
</hibernate-mapping>

```

4. Configure the child class using XML with database table

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC

```

```

"-//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
  <class name="org.techhub.onetomany.Employee" table="empdept">
    <id name="id" column="empid"/>
    <property name="name" column="empname"/>
    <property name="deptid" column="deptid" insert="false"/>
  </class>
</hibernate-mapping>

```

5. Create DAO class and insert code

Steps to create DAO class for insert one too many association data.

1. Create Parent class object and store data in it.

```

Dept d = new Dept();
d.setId(1);
d.setName("DEV");

```

2. Create Multiple Child objects and store data in it.

```

Employee emp1 = new Employee();
emp1.setId(1);
emp1.setName("RAM");

Employee emp2 = new Employee();
emp2.setId(2);
emp2.setName("SHYAM");

```

3. Create reference of collection which we use in parent class

```
Set <Employee> set=new LinkedHashSet<Employee>();
```

4. Store all child objects in set collection

```

Set <Employee> set=new LinkedHashSet<Employee>();
set.add(emp1);
set.add(emp2);

```

5. Pass collection as parameter to parent object

```
d.setEmployeeSet(set);
```

6. Save the parent object in database table using session

Complete DAO class source code.

```

package org.techhub.onetomany;
import java.util.LinkedHashSet;
import java.util.Set;
import org.hibernate.*;
import org.hibernate.cfg.*;
public class ClientApplication {
    public static void main(String[] args) {
        Configuration cfg= new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf=cfg.buildSessionFactory();
        Session session=sf.openSession();
        Transaction t=session.beginTransaction();
        Dept d = new Dept();
        d.setld(1);
        d.setName("DEV");

        Employee emp1 = new Employee();
        emp1.setld(1);
        emp1.setName("RAM");

        Employee emp2 = new Employee();
        emp2.setld(2);
        emp2.setName("SHYAM");

        Set <Employee> set=new LinkedHashSet<Employee>();
        set.add(emp1);
        set.add(emp2);

        d.setEmployeeSet(set);

        session.save(d);
        t.commit();
        session.close();
        System.out.println("Record Save Success.....");
    }
}

```

```
mysql> use hibmayjunejuly;
Database changed
mysql> select *from dept;
+-----+-----+
| deptid | deptname |
+-----+-----+
|      1 | DEV      |
+-----+-----+
1 row in set (0.03 sec)

mysql> select *from empdept;
+-----+-----+-----+
| empid | empname | deptid |
+-----+-----+-----+
|      1 | RAM     |      1 |
|      2 | SHYAM   |      1 |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

Internal Process by hibernate

Hibernate: create table dept (deptid integer not null, deptname varchar(255), primary key (deptid)) engine=InnoDB

Hibernate: create table empdept (empid integer not null, empname varchar(255), deptid integer, primary key (empid)) engine=InnoDB

Hibernate: alter table empdept add constraint FK62q6s84u6ao52adtbt527f68i foreign key (deptid) references dept (deptid)

Hibernate: select employee_.empid, employee_.empname as empname2_1_, employee_.deptid as deptid3_1_ from empdept employee_ where employee_.empid=?

Hibernate: select employee_.empid, employee_.empname as empname2_1_, employee_.deptid as deptid3_1_ from empdept employee_ where employee_.empid=?

Hibernate: insert into dept (deptname, deptid) values (?, ?)

Hibernate: insert into empdept (empname, empid) values (?, ?)

Hibernate: insert into empdept (empname, empid) values (?, ?)

Hibernate: update empdept set deptid=? where empid=?

Hibernate: update empdept set deptid=? where empid=?

Record Save Success.....

Following diagram shows how one to many work internally

```

Dept d = new Dept();
d.setId(1);
d.setName("DEV");

Employee emp1 = new Employee();
emp1.setId(1);
emp1.setName("RAM"); insert into dept (deptname, deptid) values (?, ?)

Employee emp2 = new Employee();
emp2.setId(2);
emp2.setName("SHYAM");

Set <Employee> set=new LinkedHashSet<Employee>();
set.add(emp1);
set.add(emp2);

d.setEmployeeSet(set);

session.save(d);

```

create table dept (deptid integer not null, deptname varchar(255), primary key (deptid))

deptid	deptname
1	DEV

create table empdept (empid integer not null, empname varchar(255), deptid integer, primary key (empid)) engine=InnoDB

empid	empname	deptid
1	ram	1
2	shyam	1

alter table empdept add constraint FK62q6s84u6ao52adtbtts27f68i foreign key (deptid) references dept (deptid)

insert into empdept (empname, empid) values (?, ?)

insert into empdept (empname, empid) values (?, ?)

update empdept set deptid=? where empid=?

update empdept set deptid=? where empid=?

Dept d = new Dept();

id=1 name=DEV
employeeSet

id=1 name=RAM deptid=0	id=2 name=SHYAM deptid=0
------------------------------	-----------------------------

How to Configure one too many by using annotations

If we want to implement one too many association by using annotations we have to use following annotations.

```

@OneToMany(fetch=FetchType.LAZY |
FetchType.EAGER,targetEntity=childclassname.class,cascade=CascadeType.ALL)
@JoinColumn(name="childforeigncolumnname",
referenceColumnName="parentcolumnname")

```

Q. What is fetch type in association?

FetchType is an enumerated type in the java persistence API (JPA) that specifies whether the field or property should be lazily or eagerly loaded.

There are two types of fetch type

Lazy : this is your default fetch type in hibernate. It means that the associated entities will be fetch only when it is access for the first time. This can improve performance in case where the associated entity not required more than one time.

Eager: This Fetch type will be fetch entity every time from database when main entity fetch. This will be useful when we want to associate entity every time but it may be decrease performance.

Q. what is meaning of @JoinColumn annotation?

@JoinColumn annotation in hibernate is used for the mapping foreign key column in relationship between two entities.

Example with source code

Parent class (Dept.java)

```
package org.techhub.onetomany;
import java.util.*;

import javax.persistence.CascadeType;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.FetchType;
import javax.persistence.Id;
import javax.persistence.JoinColumn;
import javax.persistence.OneToMany;
import javax.persistence.Table;
@Entity
@Table(name="annotdept")
public class Dept {
    @Id
    @Column(name="deptid")
    private int id;
    @Column(name="deptname")
    private String name;
    @OneToMany(fetch = FetchType.LAZY,targetEntity =
org.techhub.onetomany.Employee.class,cascade = CascadeType.ALL)
    @JoinColumn(name="deptid", referencedColumnName = "deptid")
    private Set<Employee> employeeSet;
    public Set<Employee> getEmployeeSet() {
        return employeeSet;
    }
    public void setEmployeeSet(Set<Employee> employeeSet) {
        this.employeeSet = employeeSet;
    }
    public int getId() {
        return id;
    }
}
```

```

        public void setId(int id) {
            this.id = id;
        }
        public String getName() {
            return name;
        }
        public void setName(String name) {
            this.name = name;
        }
    }

```

2. Child class (Employee.java)

```

package org.techhub.onetomany;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;
@Entity
@Table(name="annotemp")
public class Employee {
    @Id
    @Column(name="empid")
    private int id;
    @Column(name="empname")
    private String name;
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
}

```

```
}
```

3.hibernate.cfg.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <!-- Related to the connection START -->
        <property name="connection.driver_class">com.mysql.cj.jdbc.Driver</property>
        <property
name="connection.url">jdbc:mysql://localhost:3306/hibmayjunejuly</property>
        <property name="connection.user">root</property>
        <property name="connection.password">root</property>
        <!-- Related to the connection END -->
        <!-- Related to hibernate properties START -->
        <property name="show_sql">>true</property>
        <property name="dialect">org.hibernate.dialect.MySQL8Dialect</property>
        <property name="hbm2ddl.auto">update</property>
        <!-- Related to hibernate properties END -->
        <!-- Related to mapping START -->
        <mapping class="org.techhub.onetomany.Dept" />
        <mapping class="org.techhub.onetomany.Employee" />
    </session-factory>
</hibernate-configuration>
```

4. ClientApplication

```
package org.techhub.onetomany;
import java.util.LinkedHashSet;
import java.util.Set;

import org.hibernate.*;
import org.hibernate.cfg.*;
public class ClientApplication {
    public static void main(String[] args) {
        Configuration cfg= new Configuration();
        cfg.configure("hibernate.cfg.xml");
```

```

SessionFactory sf=cfg.buildSessionFactory();
Session session=sf.openSession();
Transaction t=session.beginTransaction();
Dept d = new Dept();
d.setId(1);
d.setName("DEV");

Employee emp1 = new Employee();
emp1.setId(1);
emp1.setName("RAM");

Employee emp2 = new Employee();
emp2.setId(2);
emp2.setName("SHYAM");

Set <Employee> set=new LinkedHashSet<Employee>();
set.add(emp1);
set.add(emp2);

d.setEmployeeSet(set);

session.save(d);
t.commit();
session.close();
System.out.println("Record Save Success.....");
    }
}

```

How to implement the many to one association using XML

Example: Multiple customers can have single vendors.

Steps.

1. Create parent class (e.g) Vendor

```
package org.techhub.manytoone;
```

```
public class Vendor {  
    private int id;  
    public int getId() {  
        return id;  
    }  
    public void setId(int id) {  
        this.id = id;  
    }  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
    private String name;  
}
```

2. Create Child class and create Parent object in child class

```
package org.techhub.manytoone;
```

```
public class Customer {  
    private int id;  
  
    public int getId() {  
        return id;  
    }  
  
    public void setId(int id) {  
        this.id = id;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

```

    }

    public Vendor getVendor() {
        return vendor;
    }

    public void setVendor(Vendor vendor) {
        this.vendor = vendor;
    }

    private String name;
    private Vendor vendor;
}

```

3. Write Mapping file for parent

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
    <class name="org.techhub.manytoone.Vendor" table="vendor">
        <id name="id" column="vendorid"/>
        <property name="name" column="vendorname"/>
    </class>
</hibernate-mapping>

```

4. Write mapping for child

customer.hbm.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
    <class name="org.techhub.manytoone.Customer" table="customer">
        <id name="id" column="custid"/>
        <property name="name" column="custname"/>
        <many-to-one name="vendor" column="vid" class="org.techhub.manytoone.Vendor"
cascade="all"/></many-to-one>
    </class>

```

</hibernate-mapping>

5. Write DAO class

Steps to write DAO in many to one association

1. Create Object of parent class

```
Vendor v= new Vendor();  
v.setId(1);  
v.setName("TATA");
```

2. Create multiple objects of child classes & set parent object as parameter to child object

```
//child objects  
Customer c1 = new Customer();  
c1.setId(1);  
c1.setName("ABC");  
c1.setVendor(v);  
  
Customer c2=new Customer();  
c2.setId(2);  
c2.setName("PQR");  
c2.setVendor(v);
```

3. Save child objects in database using session object

Complete source code of many to one association

```
package org.techhub.manytoone;
```

```
import org.hibernate.*;
```

```
import org.hibernate.cfg.*;
```

```
public class DAOAPP {
```

```
    public static void main(String[] args) {
```

```
        Configuration cfg = new Configuration();  
        cfg.configure("hibernate.cfg.xml");  
        SessionFactory sf = cfg.buildSessionFactory();  
        Session session = sf.openSession();  
        Transaction t = session.beginTransaction();  
        // parent object  
        Vendor v = new Vendor();
```

```

        v.setId(1);
        v.setName("TATA");

        // child objects
        Customer c1 = new Customer();
        c1.setId(1);
        c1.setName("ABC");
        c1.setVendor(v);

        Customer c2 = new Customer();
        c2.setId(2);
        c2.setName("PQR");
        c2.setVendor(v);

        session.save(c1);
        session.save(c2);
        t.commit();
        session.close();
        System.out.println("Success.....");
    }
}

hibernate.cfg.xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <!-- Related to the connection START -->
        <property name="connection.driver_class">com.mysql.cj.jdbc.Driver</property>
        <property
name="connection.url">jdbc:mysql://localhost:3306/hibmayjunejuly</property>
        <property name="connection.user">root</property>
        <property name="connection.password">root</property>
        <!-- Related to the connection END -->
        <!-- Related to hibernate properties START -->
        <property name="show_sql">>true</property>

```



```

        <property name="dialect">org.hibernate.dialect.MySQL8Dialect</property>
        <property name="hbm2ddl.auto">update</property>
        <!-- Related to hibernate properties END -->
        <!-- Related to mapping START -->
        <mapping resource="vendor.hbm.xml" />
        <mapping resource="customer.hbm.xml" />
    </session-factory>
</hibernate-configuration>

```

How to use many to one using annotations

1. Parent class (vendor.java)

```
package org.techhub.manytoone;
```

```

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;
@Entity
@Table(name="annotvendor")
public class Vendor {
    @Id
    @Column(name="vid")
    private int id;
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    @Column(name="vname")
    private String name;
}

```

```
}
```

2. Customer.java

```
package org.techhub.manytoone;
```

```
import javax.persistence.CascadeType;
```

```
import javax.persistence.Column;
```

```
import javax.persistence.Entity;
```

```
import javax.persistence.Id;
```

```
import javax.persistence.JoinColumn;
```

```
import javax.persistence.ManyToOne;
```

```
import javax.persistence.Table;
```

```
import org.hibernate.annotations.ManyToAny;
```

```
@Entity
```

```
@Table(name="annotcustomer")
```

```
public class Customer {
```

```
    @Id
```

```
    @Column(name="custid")
```

```
    private int id;
```

```
    public int getId() {
```

```
        return id;
```

```
    }
```

```
    public void setId(int id) {
```

```
        this.id = id;
```

```
    }
```

```
    public String getName() {
```

```
        return name;
```

```
    }
```

```
    public void setName(String name) {
```

```
        this.name = name;
```

```
    }
```

```
    public Vendor getVendor() {
```

```

        return vendor;
    }

    public void setVendor(Vendor vendor) {
        this.vendor = vendor;
    }

    @Column(name="custname")
    private String name;
    @ManyToOne(cascade = CascadeType.ALL)
    @JoinColumn(name="vendorid", referencedColumnName = "vid")
    private Vendor vendor;
}

```

3.DAOAPP.java

```

package org.techhub.manytoone;
import org.hibernate.*;
import org.hibernate.cfg.*;
public class DAOAPP {
    public static void main(String[] args) {
        Configuration cfg = new Configuration ();
        cfg.configure ("hibernate.cfg.xml");
        SessionFactory sf = cfg.buildSessionFactory ();
        Session session = sf.openSession ();
        Transaction t = session.beginTransaction();
        // parent object
        Vendor v = new Vendor ();
        v.setId (1);
        v.setName ("TATA");

        // child objects
        Customer c1 = new Customer ();
        c1.setId (1);
        c1.setName ("ABC");
        c1.setVendor (v);

        Customer c2 = new Customer();
        c2.setId(2);
        c2.setName("PQR");
    }
}

```

```

        c2.setVendor(v);

        session.save(c1);
        session.save(c2);

        t.commit ();
        session.close();
        System.out.println ("Success.....");
    }
}

```

Many too many associations

How to implement the many to many associations using XML

1. Create Parent class with collection property

```

package org.techhub.manytomany;
import java.util.*;
public class Vendor {
    private int id;
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public Set<Customer> getCustomers() {
        return customers;
    }
    public void setCustomers(Set<Customer> customers) {
        this.customers = customers;
    }
    private String name;
}

```

```
        private Set<Customer> customers;
    }
}
```

2. Create Child class with collection property

```
package org.techhub.manytomany;
```

```
import java.util.*;

public class Customer {
    private int id;

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public Set<Vendor> getVendors() {
        return vendors;
    }

    public void setVendors(Set<Vendor> vendors) {
        this.vendors = vendors;
    }

    private String name;
    private Set<Vendor> vendors;
}
}
```

3. Write parent mapping file for provide mapping

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
"-//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">

<hibernate-mapping>
    <class name="org.techhub.manytomany.Vendor" table="vendorsmany">
        <id name="id" column="vendorid" />
        <property name="name" column="vendorname" />
        <set name="customers" inverse="false" cascade="all"
            table="vendor_customer">
            <key column="vendorid" />
            <many-to-many class="org.techhub.manytomany.Customer"
column="custid" />
        </set>
    </class>
```

```
</hibernate-mapping>
```

4. Write child mapping file for provide mapping

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
"-//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">

<hibernate-mapping>
    <class name="org.techhub.manytomany.Customer" table="customermany">

        <id name="id" column="custid" />
        <property name="name" column="custname" />
        <set name="vendors" cascade="all" table="vendor_customer">
            <key column="custid" />
            <many-to-many class="org.techhub.manytomany.Vendor"
column="vendorid" />
        </set>
    </class>
</hibernate-mapping>
```

5. Write DAO class

If we want to write DAO class we have steps.

1. Create multiple object of parent class.

```
Vendor v1 = new Vendor();  
v1.setId(1);  
v1.setName("TATA");
```

```
Vendor v2 = new Vendor();  
v2.setId(2);  
v2.setName("BAJAJ");
```

2. Create multiple objects of child class

```
Customer c1 = new Customer();  
c1.setId(1);  
c1.setName("ABC");
```

```
Customer c2 = new Customer();  
c2.setId(2);  
c2.setName("PQR");
```

3. Create Reference of Collection and store child objects in it.

```
Set<Customer> customerSet = new LinkedHashSet<Customer>();  
customerSet.add(c1);  
customerSet.add(c2);
```

4. Set Collection to every parent object.

```
v1.setCustomers(customerSet);  
v2.setCustomers(customerSet);
```

5. Save the parent objects in database using session object

Complete DAO object App

```
package org.techhub.manytomany;  
import org.hibernate.*;  
import org.hibernate.cfg.*;  
import java.util.*;  
public class DAOAPP {  
    public static void main(String[] args) {
```

```
Configuration cfg = new Configuration();
cfg.configure("hibernate.cfg.xml");
SessionFactory sf = cfg.buildSessionFactory();
Session session = sf.openSession();
Transaction t = session.beginTransaction();
```

```
Vendor v1 = new Vendor();
v1.setId(1);
v1.setName("TATA");
```

```
Vendor v2 = new Vendor();
v2.setId(2);
v2.setName("BAJAJ");
```

```
Customer c1 = new Customer();
c1.setId(1);
c1.setName("ABC");
```

```
Customer c2 = new Customer();
c2.setId(2);
c2.setName("PQR");
```

```
Set<Customer> customerSet = new LinkedHashSet<Customer>();
customerSet.add(c1);
customerSet.add(c2);
```

```
v1.setCustomers(customerSet);
v2.setCustomers(customerSet);
```

```
session.save(v1);
session.save(v2);
```

```
t.commit();
session.close();
```

```
}
```

```
}
```

How implement many to many by using annotations

@ManyToOne(targetEntity=classname.class,cascade=CascadeType.ALL)


```
@JoinTable(name="jointablename",joinColumns=@JoinColumn(name="foreignkeycol",referencedColumnName="catid"),inverseJoinColumns=@JoinColumn(name="item_id_fk",referencedColumnName="itemid"))
```

Example with source code

```
package org.techhub.manytomany;
import java.util.*;
import javax.persistence.CascadeType;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.ForeignKey;
import javax.persistence.Id;
import javax.persistence.JoinColumn;
import javax.persistence.JoinTable;
import javax.persistence.ManyToMany;
import javax.persistence.Table;
@Entity
@Table(name="vendormany")
public class Vendor {

    @Id
    @Column(name="vid")
    private int id;
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public Set<Customer> getCustomers() {
        return customers;
    }
}
```

```

        public void setCustomers(Set<Customer> customers) {
            this.customers = customers;
        }
        @Column(name="vendname")
        private String name;
        @ManyToMany(targetEntity=org.techhub.manytomany.Customer.class,cascade=Cascad
eType.ALL)
        @JoinTable(name="vendcustjoin", joinColumns =@JoinColumn(name="vid",
referencedColumnName = "vid"), inverseJoinColumns = @JoinColumn(name="cust",
referencedColumnName = "custid"))
        private Set<Customer> customers;
    }

```

Customer.java

```

package org.techhub.manytomany;
import java.util.*;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.JoinColumn;
import javax.persistence.ManyToMany;
import javax.persistence.Table;
@Entity
@Table(name="customermany")
public class Customer {
    @Id
    @Column(name="custid")
    private int id;
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
}

```

```

    public void setName(String name) {
        this.name = name;
    }

    public Set<Vendor> getVendors() {
        return vendors;
    }

    public void setVendors(Set<Vendor> vendors) {
        this.vendors = vendors;
    }
@Column(name="custname")
    private String name;

@ManyToMany(targetEntity=org.techhub.manytomany.Vendor.class,mappedBy="customers")
    private Set<Vendor> vendors;
}

```

hibernate.cfg.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <!-- Related to the connection START -->
        <property name="connection.driver_class">com.mysql.cj.jdbc.Driver</property>
        <property
name="connection.url">jdbc:mysql://localhost:3306/hibmayjunejuly</property>
        <property name="connection.user">root</property>
        <property name="connection.password">root</property>
        <!-- Related to the connection END -->
        <!-- Related to hibernate properties START -->
        <property name="show_sql">>true</property>
        <property name="dialect">org.hibernate.dialect.MySQL8Dialect</property>
        <property name="hbm2ddl.auto">update</property>
        <!-- Related to hibernate properties END -->
        <!-- Related to mapping START -->
        <mapping class="org.techhub.manytomany.Vendor" />
    
```

```
        <mapping class="org.techhub.manytomany.Customer" />
    </session-factory>
</hibernate-configuration>
```

DAOAPP.java

```
package org.techhub.manytomany;
import org.hibernate.*;
import org.hibernate.cfg.*;
import java.util.*;

public class DAOAPP {
    public static void main(String[] args) {
        Configuration cfg = new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf = cfg.buildSessionFactory();
        Session session = sf.openSession();
        Transaction t = session.beginTransaction();

        Vendor v1 = new Vendor();
        v1.setId(1);
        v1.setName("TATA");

        Vendor v2 = new Vendor();
        v2.setId(2);
        v2.setName("BAJAJ");

        Customer c1 = new Customer();
        c1.setId(1);
        c1.setName("ABC");

        Customer c2 = new Customer();
        c2.setId(2);
        c2.setName("PQR");

        Set<Customer> customerSet = new LinkedHashSet<Customer>();
        customerSet.add(c1);
        customerSet.add(c2);

        v1.setCustomers(customerSet);
```

```

        v2.setCustomers(customerSet);

        session.save(v1);
        session.save(v2);

        t.commit();
        session.close();
    }
}

```

How to write Left join using Hibernate

if we want to perform left or any join using hibernate we have use following syntax

```

select pref.parentfield, cref.childfield from parentclassname ref join type
parentref.collectionname ref

```

Example with source code

```

package org.techhub.manytomany;
import org.hibernate.*;
import org.hibernate.cfg.*;
import java.util.*;
public class LeftJoinApplication {

    public static void main(String[] args) {
        Configuration cfg =new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf=cfg.buildSessionFactory();
        Session session=sf.openSession();
        Transaction t=session.beginTransaction();
        Query q=session.createQuery("select v.name, c.name from Vendor v left join
v.customers c");
        List<Object[]> list=q.list();
        for(Object obj[]:list) {
            System.out.println(obj[0]+"\\t"+obj[1]);
        }
    }
}

```

Example of right join using hibernate

```
package org.techhub.manytomany;
import org.hibernate.*;
import org.hibernate.cfg.*;
import java.util.*;
public class RightJoinApplication {

    public static void main(String[] args) {
        Configuration cfg =new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf=cfg.buildSessionFactory();
        Session session=sf.openSession();
        Transaction t=session.beginTransaction();
        Query q=session.createQuery("select v.name, c.name from Vendor v right join
v.customers c");
        List<Object[]> list=q.list();
        for(Object obj[]:list) {
            System.out.println(obj[0]+"\\t"+obj[1]);
        }
    }
}
```

Example with source code

```
package org.techhub.manytomany;
import org.hibernate.*;
import org.hibernate.cfg.*;
import java.util.*;
public class LeftJoinApplication {

    public static void main(String[] args) {
        Configuration cfg =new Configuration();
        cfg.configure("hibernate.cfg.xml");
        SessionFactory sf=cfg.buildSessionFactory();
        Session session=sf.openSession();
        Transaction t=session.beginTransaction();
        Query q=session.createQuery("select v.name, c.name from Vendor v inner join
v.customers c");
```

```

        List<Object[]> list=q.list();
        for(Object obj[]:list) {
            System.out.println(obj[0]+"\\t"+obj[1]);
        }
    }
}

```

What are some of the important interfaces of the Hibernate framework?

1. **The SessionFactory** creates and manages the sessions that are occurring over a short conversation between your program and the database.
2. **The session** object provides an interface between the application and data stored in the database. It holds the first level cache(mandatory) of data. Sessions are the most important interfaces in the hibernate framework that gives you a tool like adding the data, changing the data, or getting data from the database and once you are done with the session, you need to close this session.
3. **Transaction interface** represents a database transaction including the committing changes to the database or the rollback.
4. **The query interface** is used for creating and executing the queries and retrieving the information from the database.
5. **SessionFactoryBuilder** is used to build a SessionFactory instance based on configuration settings for a new SessionFactory.

List some of the databases supported by Hibernate.

1. MySQL
2. Oracle
3. PostgreSQL
4. H2
5. DB2
6. Microsoft SQL Server
7. Sybase
8. SQLite

What is a Session in Hibernate?

Session represents the single unit of work that acts as a gateway for interacting with the databases, Hibernate session is the primary interface for working with databases. Session provides various important functions such as

- Transaction Management

- Caching
- Lazy Loading
- Data Retrieval Strategies
- Object Persistence and Retrieval

It also handles Data Relationship Handling where developers can manage and define relationships between objects, specifying fetching strategies, and overflow of behaviors.

What is SessionFactory?

SessionFactory in Hibernate is to create and manage the *session* instances that-

- configure the management by reading
- managing the configuration setting of the hibernate environment
- database connection
- mapping metadata (the data inside the data called metadata)
- caching configurations.

When you create the *SessionFactory* it is shared among multiple threads within the application where each thread acquires its own *Session* instance shared from the *SessionFactory*.

NOTE: *SessionFactory is thread-safe but Sessions is not thread-safe.*

What is Hibernate caching?

It is a technique which reduces repeatedness of fetching data from the database which clean application response timing, reduces load on database server and improves performance of the database communication by saving frequent data in memory.

Hibernate provides many levels of caching where each serves different benefits and specific purposes. It could efficiently improve application performance, whenever complex queries are involved or the same data access again and again.

Types of Hibernate Caching:

I. First-Level(Session) Cache:

- It is enabled by default and cache mapped with Hibernate *Session*.
- It stores objects of the current session which are recovered.
- It also provides data consistency at transaction level.

II. Second-Level(Session Factory) Cache:

- In the same *SessionFactory*, the cache is distributed among all the sessions created.
- It caches data used in different sessions.
- It also provides data consistency at application level.

Can we declare the Entity class final?

Yes, you can declare an entity or class as 'final' in hibernate but it has some complications as it can't be subclassed which affects hibernate's ability to perform tasks like runtime enhancements and proxy generation for lazy loading for certain features.

3. What are the disadvantages of using Hibernate?

Hibernate also has some disadvantages, such as:

- **Learning curve:** *Hibernate is a complex framework, so it takes some time to learn how to use it effectively.*
- **Performance:** *Hibernate can add some overhead to database access, so it is important to use it carefully to avoid performance problems.*
- **Vendor lock-in:** *Hibernate is an open source project, but it is developed and maintained by a single vendor. This can make it difficult to switch to a different ORM framework if necessary.*

4. What are some of the important interfaces of Hibernate framework?

Some of the important interfaces of Hibernate framework are:

- **SessionFactory:** *This is the factory class that is used to create Session objects.*
- **Session:** *This is the main object used to interact with the database.*
- **Transaction:** *This is used to group a series of database operations together.*
- **Criteria:** *This is used to create dynamic queries.*
- **Query:** *This is used to execute a predefined query.*
- **HQL:** *This is a high-level query language that is used to query the database.*

5. What is lazy loading in Hibernate?

Lazy loading is a technique that is used to defer the loading of an object's child objects until they are actually needed. This can improve performance by reducing the number of database queries that need to be executed.

Lazy loading is enabled by default in Hibernate. However, it can be disabled for specific objects or relationships.

6. What is the difference between first level cache and second level cache?

First level cache is a cache that is associated with a Session object. It is used to store objects that have been loaded by the Session.

Second level cache is a shared cache that is used to store objects that have been loaded by multiple Session objects. It can be used to improve performance by reducing the number of database queries that need to be executed.

7. What are some of the best practices for using Hibernate?

Some of the best practices for using Hibernate include:

- **Use immutable objects:** *Immutable objects are objects that cannot be modified after they are created. This can help to improve performance and prevent data corruption.*

- **Use lazy loading:** Lazy loading can improve performance by deferring the loading of an object's child objects until they are actually needed.
- **Use caching:** Caching can improve performance by storing frequently accessed objects in memory.
- **Use transactions:** Transactions can help to ensure the ACID properties of database operations.
- **Use logging:** Logging can help to troubleshoot problems with Hibernate applications.

8. How is SQL query created in Hibernate?

In Hibernate, SQL queries are generated automatically based on HQL (Hibernate Query Language), these are generated from high-level queries and are autonomously produced or through the criteria API. The Hibernate translator takes these high-level queries and converts them into the actual SQL queries so that the database could understand. This operation is referred to as the query translation. This process gives developers the ability to engage in the database interaction through queries resembling Java that constructs while the underlying translation to SQL is executed seamlessly.

15. What is the difference between session.save() and session.persist() method?

Properties	session.save()	session.persist()
Return Value	It will give a generated identifier.	It will not generate anything.
Identifier Generation	It is necessary for non-assigned IDs.	It is not necessary and can be assigned directly.
Transient Instances	It will be detached or transient instance.	It should be a Transient instance.
Cascade Operations	It can't cascade on its own.	It can be cascaded.
Managed State	It will give the controlled instance.	It will be rest as transient.
Flush Immediate	It will give you an instant SQL INSERT.	It will not have instant SQL operation.
Unsaved Transient State	It is not secured with unsaved transient instances.	It is secured with unsaved transient instances.
Inverse Associations	It will be cascaded to reverse end of associations.	It will not be cascaded to the reverse end of associations.

7. What is the difference between update and merge method?

Properties	update()	merge()
Managed Object	It should be a continuing object.	It should be a detached or transient object.
Return Type	It gives a void object.	It gives a managed object.
Overwrite Detached	It will reinitialize the detached object.	It will replicate the states to structured objects.
Unsaved Transient	It will throw an exception for the transient.	It will carry on the transient object.
Copying State	It will replicate all the states covering nulls.	It will only replicate the null states.
Cascade Operations	It will be cascaded to connected entities.	It will be cascaded through required entity mapping.
Stale Data Handling	It will be upgraded to its current state.	It will control the old data conflict.
Versioned Entities	It will need the version property.	It will control the version property.

18. What is the difference between first-level cache and second-level cache?

Properties	First-Level Cache	Second-Level Cache
Scope	It will be restricted to the Hibernate Session.	It will be divided into processes and sessions.
Granularity	It will give entity-level results.	It will give query and entity-level results.
Storage	It will reside in memory.	It will be configurable as memory, disk, etc.
Concurrency	It's not suggested for multi-session concurrency.	It's mostly used for multi-session concurrency.
Lifetime	It will only stay for the session's timings.	It could have configurable lifetimes.
Types of Objects Cached	It has Structured(persistent) objects cache.	It has queries, collections, structured objects, etc.
Cache Strategies	It has the in-built (not user-configurable) strategies.	It has the user-defined strategies.
Customization	It will provide restricted customize options.	It will provide you with more regulated and customized options.
Cache Providers	It has only a single for each session factory.	It might have many session factory providers.
Use Cases	It will avail you of the performance enhancement inside the session.	It will avail you the performance enhancement over processes and sessions.

9. Can you tell the difference between `getCurrentSession()` and `openSession()` methods?

Properties	<code>getCurrentSession()</code>	<code>openSession()</code>
Life Management	It will be handled by Hibernate and bind to the transaction.	It will be handled by the application and have to be manually structured.
Scope	It is only restricted to current transactions.	It is not restricted to any particular transaction.
Automatic Closure	It will automatically terminate at the end of the transaction.	It should be closed by hand.
Exception Handling	Here hibernate controls the exception propagation.	Here application should control the exceptions.
Configuration	It will have configuration-handle behavior.	Its explicit nature depends on how it is used.
Reusability	It will be used again and again only in the same transactions.	It will be used in different transactions also.
Contextual Awareness	It must be known in the present transaction context.	It is not known to any of the transaction context.
Use Cases	It is mostly suitable for short-lived operations.	It is mostly suitable for custom scenes or long-lived operations.

20. Differentiate between save () and saveOrUpdate() methods in the hibernate session.

Properties	session.save()	session.saveOrUpdate()
New Entity	It will be available for only transient entities.	It will be available for detached as well as transient entities.
Persistent Entity	It will throw the exception if the entity presents in DB.	It will modify the entity if available in DB apart from saving it.
Entity State	It will be the same as transient after being saved.	It will be the same as persistent after being saved.
Cascade Operations	It must be cascaded to linked entities.	It must be cascaded to entity mapping accordingly.
Identifier Strategy	It should always use ID creation for auto-increment.	It supports multiple ID identifier tactics.
Managed State	It didn't influence the structured state.	It can reconnect to the detached entity.
Unsaved Transient State	It perseveres the transient entity.	It perseveres the transient entity.
Auto-generated ID	It always needed ID creation tactics.	The ID creation tactics may be used here.
Use Cases	It will generate the new entities.	It will generate new or modify available entities.

5. What is the difference between the get and load method?

Properties	get()	load()
Return Type	It will give the real object.	It will give the substitute object.
Database Access	It will continuously strike the database.	It might not continuously strike the database.
Exception on Missing Object	It will give you 'null' if it's not available.	It will give you 'ObjectNotFoundException' if it's not available.
Proxy Initialization	It will instantly prepare the proxy.	It will slowly prepare the proxy.
Eager Fetching	It will keep up eager collecting.	It will not have eager collecting.
Unlimited State	It will give an uninitialized proxy.	It will also give a give uninitialized proxy.
Lazy Initialization Exception	It might be given if associations have not been collected.	It might give if associations have accessed outside sessions.
Use Cases	It will be used for recovering the objects when a particular existence is not known.	It will be used for slow and structured loading.

What is persistent class?

The entire concept of Hibernate is to take the values from Java class attributes and persist them to a database table. A mapping document helps Hibernate in determining how to pull the values from the classes and map them with table and associated fields.

Java classes whose objects or instances will be stored in database tables are called persistent classes in Hibernate. Hibernate works best if these classes follow some simple rules, also known as the Plain Old Java Object (POJO) programming model.